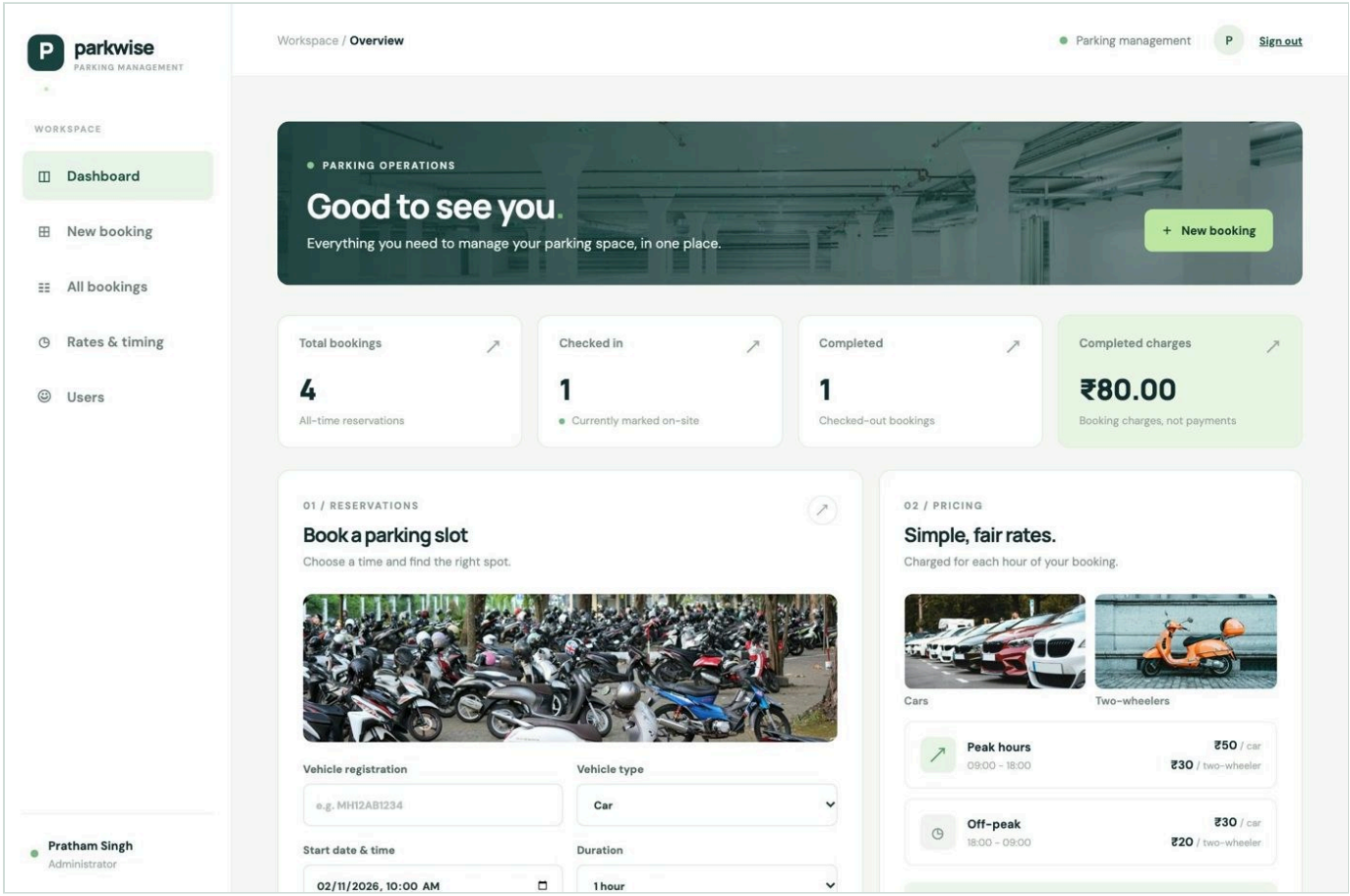
Mini Project Report

Parkwise

Parking Management Web Application with Login and User Management



Student name	Pratham Singh
PRN	1032211384
Course name	Mini Project

Abstract

Parkwise is a local web application for managing five parking slots. Users sign in, then check availability, book a slot, search bookings, view charges, check vehicles in and out, cancel, extend and open a printable receipt. An administrator can create, view, edit and delete user accounts.

It uses Java 17 and Spring Boot 3.2.5 on the server, SQLite for storage, and plain HTML, CSS and JavaScript in the browser. On the final build, all 171 API checks and 34 screen checks passed. It is a local prototype with no online payment.

5	2	18	205
Parking slots	Database tables	API routes	Checks passed

Contents

1. Project Overview 3
2. Folder and File Structure 4
3. Architecture and Technology Stack 5
4. Database Design 6
5. Java Functions 7
6. JavaScript Functions and Event Handlers 10
7. Variables and Their Roles 13
8. API Endpoints and Communication 15
9. Application Flows 17
10. Login and User Management 19
11. Running the Project 21
12. Testing 22
13. Limitations 23
14. Conclusion 23

1. Project Overview

Problem

Tracking parking by hand makes it easy to double-book a slot, miscalculate a charge or lose track of which vehicles are on site. Anyone with the records can also change them.

Solution

Parkwise gives signed-in users one dashboard for five slots. It checks conflicts, calculates charges and keeps each booking's status in a database.

Objectives

Secure access

Require sign-in, with administrator and staff roles

User management

Create, read, update and delete user accounts

Availability

Show which slots are free for a chosen time

No double-booking

Prevent overlapping bookings for a slot or a vehicle

Pricing

Apply peak, off-peak and long-stay rates

Records

Search, check in, check out, cancel and extend, with receipts

Scope

In scope	Outside the project
Five slots shared by cars and two-wheelers	Separate slots for each vehicle type
Bookings of 1 to 5 hours, on a future whole hour, up to 90 days ahead	Part-hour bookings
Booking charges calculated and displayed	Online payment, refunds or late fees
Sign-in with Administrator and Staff roles on one computer	Password reset by email and public hosting

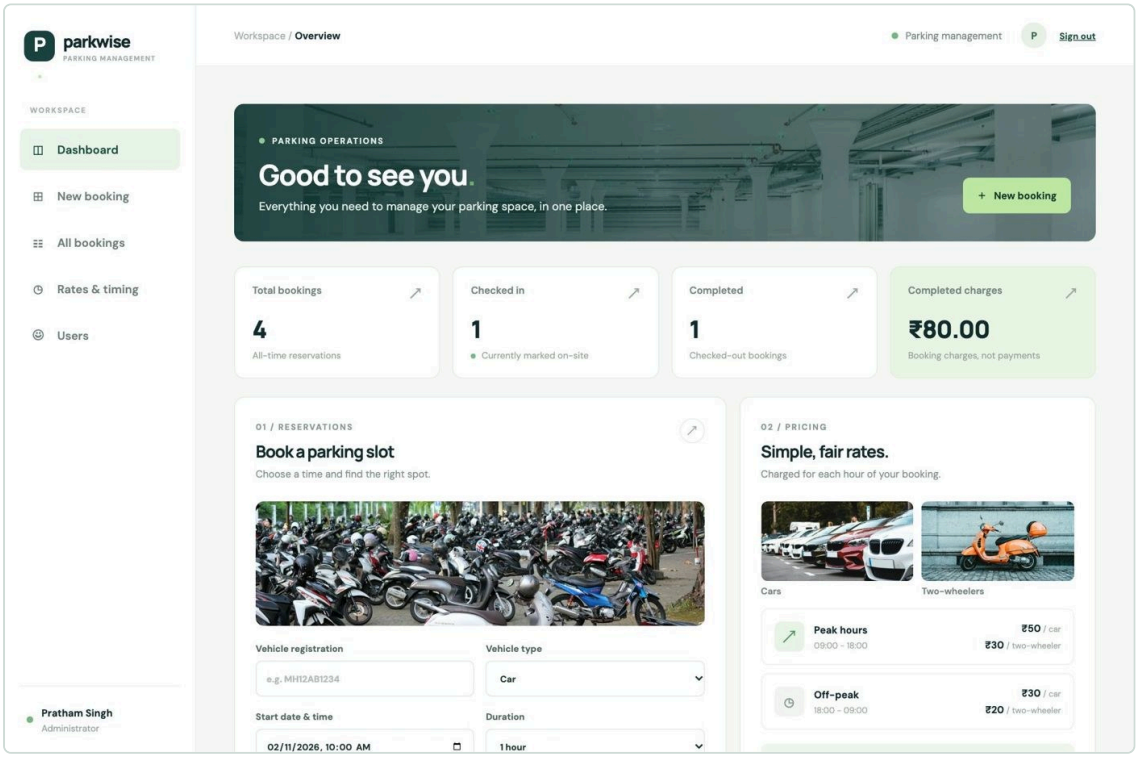


Figure 1: Parkwise dashboard from the running application

2. Folder and File Structure

The project has one Java package with three classes, three web files, five images and one test class. The parking.db file is created at run time.

```
ParkwiseWeb/
|-- .gitignore
|-- README.txt
|-- parkwise.jar
|-- pom.xml
`-- src/
    |-- main/
    |   |-- java/com/parkwise/
    |   |   |-- AuthController.java
    |   |   |-- AuthFilter.java
    |   |   |-- ParkingApplication.java
    |   |   `-- ParkingController.java
    |   `-- resources/
    |       |-- application.properties
    |       `-- static/
    |           |-- app.js
    |           |-- images/ (5 .jpg photos)
    |           |-- index.html
    |           `-- style.css
    `-- test/java/com/parkwise/
        `-- ParkingControllerTest.java
```

File	Size	Purpose
README.txt	48 lines	How to run the project, sign in, features, rates and limitations
parkwise.jar	35.0 MB	Packaged application with its libraries; runs without Maven
pom.xml	13 lines	Maven build file: Spring Boot parent, web, JDBC, SQLite and test dependencies
AuthController.java	226 lines	Sign-in, password hashing and the user CRUD endpoints
AuthFilter.java	52 lines	Requires a signed-in user on the API and refuses other websites
ParkingApplication.java	11 lines	Entry point that starts Spring Boot
ParkingController.java	177 lines	Booking logic: REST endpoints, validation, pricing and SQL
application.properties	9 lines	Database URL, pool size, error messages, cookie and server address
index.html	87 lines	Sign-in screen, sidebar, dashboard, booking form, rates, records, users and receipt
style.css	11.2 KB	Compressed page styling: layout, colours, fonts and image sizes
app.js	204 lines	All browser logic: API calls, sign-in, page updates and event handlers
images/	5 files	Unsplash photos: parking-hall, scooters-row, cars-row, scooter and parking-lot-aerial
ParkingControllerTest.java	60 lines	Two JUnit tests using an in-memory SQLite database

Configuration settings

Setting	Value	Effect
spring.datasource.url	jdbc:sqlite:parking.db	Uses parking.db in the folder the app is started from
spring.datasource.driver-class-name	org.sqlite.JDBC	Selects the SQLite JDBC driver
spring.datasource.hikari.maximum-pool-size	1	Allows one database connection in the pool
server.error.include-message	always	Includes validation messages in error responses
spring.jackson.deserialization.accept-float-as-int	false	Refuses decimal values such as 1.5 hours
server.servlet.session.cookie.same-site	lax	Stops other websites sending the sign-in cookie
server.address	127.0.0.1	Accepts connections from this computer only

3. Architecture and Technology Stack

The browser page talks to two Spring Boot controllers. A filter checks every API request first, and both controllers use one SQLite file.

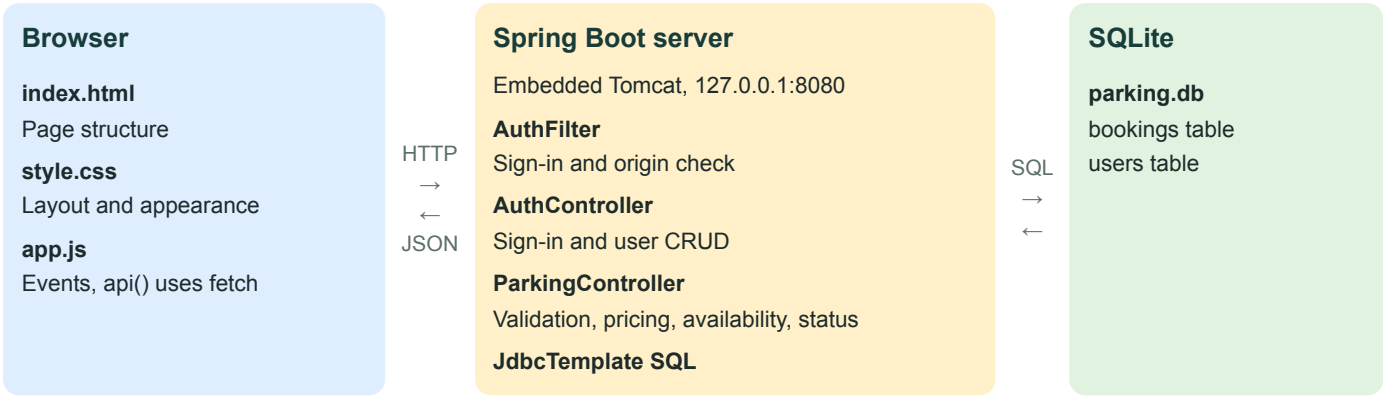


Figure 2: System architecture

Technology stack

Layer	Technology	Use in this project
Language	Java 17	Back-end code; java.version is 17 in pom.xml
Framework	Spring Boot 3.2.5	Starts the server, handles REST routes and converts JSON
Web server	Embedded Tomcat 10.1.20	Included in the jar; also holds the sign-in sessions
Security	PBKDF2 with HMAC-SHA256	Hashes passwords with a salt, using Java's own crypto library
Data access	Spring JDBC, JdbcTemplate	Runs parameterized SQL statements
Database	SQLite, sqlite-jdbc 3.45.3.0	Stores bookings and users in one local file
Front end	HTML, CSS, JavaScript	One page with no framework, plus five local photos
Fonts	DM Sans and Manrope	Imported from Google Fonts in style.css, which needs internet access
Testing	JUnit 5 through Spring Boot test starter	Tests the controller directly with in-memory SQLite
Build	Maven, Spring Boot plugin	Builds the runnable jar

Key design choices

Parameterized SQL
User values go through ? placeholders

Safe display
escapeHtml() protects text placed into the page

Text dates
yyyy-MM-dd HH:mm sorts and compares correctly

Hashed passwords
Only a salted hash is stored, never the password

Checked per request
The filter rereads the user, so deleting one ends their session

Server decides
Every rule is enforced on the server, not only in the page

4. Database Design

Parkwise uses two tables. initialize() in each controller creates its table at start-up, so an older parking.db is upgraded in place. The two tables are not linked by a foreign key.

bookings table

No.	Field	Type	Rules	Purpose
1	id	INTEGER	Primary key, auto increment	Unique booking number
2	car_number	TEXT	NOT NULL	Upper-case vehicle registration, for cars and two-wheelers
3	slot	INTEGER	NOT NULL	Slot number; the controller allows 1 to 5
4	start_time	TEXT	NOT NULL	Reserved start, as yyyy-MM-dd HH:mm
5	end_time	TEXT	NOT NULL	Reserved end, as yyyy-MM-dd HH:mm
6	hours	INTEGER	NOT NULL	Total booked hours; the controller allows 1 to 5
7	price	REAL	NOT NULL	Booking charge in rupees
8	status	TEXT	NOT NULL	BOOKED, CHECKED_IN, CHECKED_OUT or CANCELLED
9	vehicle_type	TEXT	NOT NULL, default 'CAR'	CAR or TWO_WHEELER
10	checked_in_at	TEXT	Optional	Time of check-in, set by the server
11	checked_out_at	TEXT	Optional	Time of check-out, set by the server

users table

No.	Field	Type	Rules	Purpose
1	id	INTEGER	Primary key, auto increment	Unique user number
2	username	TEXT	NOT NULL, UNIQUE, NOCASE	Sign-in name; ignores upper and lower case
3	full_name	TEXT	NOT NULL	Name shown in the page
4	password_hash	TEXT	NOT NULL	Salted PBKDF2 hash; never returned by the API
5	role	TEXT	NOT NULL	ADMIN or STAFF
6	created_at	TEXT	NOT NULL	When the account was created

Allowed slot, status, vehicle and role values are enforced by Java code, not by database constraints. Records with no vehicle type default to CAR.

Booking status changes

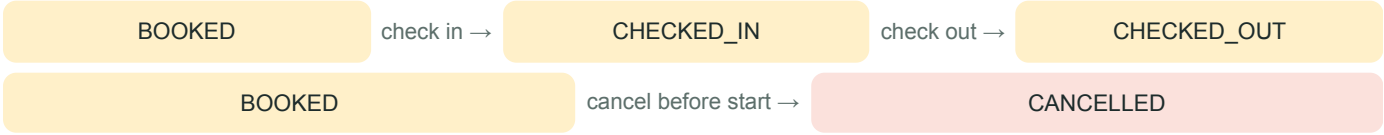


Figure 3: Status changes allowed by the controller

Extension does not change the status. It updates end_time, hours and price. The application never deletes a booking record.

5. Java Functions

ParkingController handles bookings, and AuthController handles sign-in and users. Their endpoint paths begin with /api.

Application start-up

Function	Inputs	Output	Purpose
ParkingApplication.main	String[] args	None	Calls SpringApplication.run to start the application and server
ParkingController(db) constructor	JdbcTemplate db	Controller object	Receives the database helper through Spring dependency injection
initialize()	None	None	Runs after construction through @PostConstruct; creates the table and adds missing columns
addColumn(column, type)	String column, String type	None	Reads PRAGMA table_info and adds the column if it is missing

ParkingController helper functions

Function	Inputs	Output	Purpose
bad(message)	String message	ResponseStatusException	Builds an HTTP 400 error with the given message
now()	None	String	Returns the current local time in FORMAT
parseTime(text)	String text	LocalDateTime	Replaces T with a space and parses strictly; raises 400 if invalid or null
futureHour(text)	String text	LocalDateTime	Rejects a time unless it is a future whole hour within 90 days
registration(value)	String value	String	Trims and upper-cases; allows letters, numbers, spaces and hyphens up to 30 characters
compact(registration)	String registration	String	Removes spaces and hyphens so two spellings match
vehicleFree(registration, start, end, excludeId)	String, two LocalDateTime, int	boolean	Returns true when the vehicle has no other active booking in that time
type(value)	String value	String	Returns the value if it is CAR or TWO_WHEELER; otherwise raises 400
hours(value)	int value	None	Raises 400 unless the duration is from 1 to 5
price(start, hours, type)	LocalDateTime, int, String	double	Adds the rate for each hour, then applies 10% off for 4 or more hours
available(slot, start, end, excludeId)	int, two LocalDateTime, int	boolean	Returns true when no active booking conflicts with the time range
find(id)	int id	Map<String, Object>	Returns one booking row, or raises 404
integer(value), string(value)	Object value	int, String	Convert database values; string() uses an empty string for null

Request data records

Record	Fields	Purpose
BookingRequest	registration, vehicleType, startTime, hours, slot	Holds the JSON body for a new booking
ExtensionRequest	extraHours	Holds the JSON body for an extension
LoginRequest	username, password	Holds the JSON body for sign-in
UserRequest	username, fullName, password, role, currentPassword	Holds the JSON body for setup and for adding or editing a user

Booking endpoint functions

Function	Inputs	Output	Purpose
availability(...)	Query: startTime, hours, vehicleType	Map with slots, price, endTime	Validates the input, checks slots 1 to 5, and returns the estimate and end time
book(request)	BookingRequest JSON body	New booking row	Validates all fields, rechecks the slot and vehicle, inserts the row and returns it
bookings(search, by)	Query: search, by; both optional	List of booking rows	Returns all bookings or filters by part of a registration, ID or slot, newest first
booking(id)	Path: id	One booking row	Returns one booking through find()
action(id, action)	Path: id, action	Updated booking row	Runs cancel, check-in or check-out when the rules allow it
extend(id, request)	Path: id; ExtensionRequest	Updated booking row	Validates the extra hours, checks the added time and updates end, hours and price
statistics()	None	Map of totals	Returns total, cancelled, checked_in, completed, completed_charges and outstanding_charges

AuthController functions

Function	Inputs	Output	Purpose
initialize()	None	None	Creates the users table and the decoy hash at start-up
hash(password)	String password	String	Makes a random 16-byte salt and returns a stored PBKDF2 hash
derive(password, salt, iterations)	String, byte[], int	byte[]	Runs PBKDF2WithHmacSHA256 for 210,000 iterations
matches(password, stored)	String, String	boolean	Compares a password with a stored hash in constant time
username(), fullName(), password(), role()	String value	String	Validate each field; raise 400 if invalid
unique(username, excludeId)	String, int	None	Raises 409 if another account uses the name
admins()	None	int	Counts administrators, to protect the last one
publicUser(id)	int id	Map	Returns a user without the password hash, or raises 404
signedIn(request)	HttpServletRequest	Map	Reads the user placed on the request by AuthFilter
startSession(request, id)	HttpServletRequest, int	None	Replaces the old session and stores the user id and a password stamp

Function	Inputs	Output	Purpose
sessionUser(db, session)	JdbcTemplate, HttpSession	Map or null	Rereads the user; returns null if deleted or the password changed
status(request)	HttpServletRequest	Map	Reports whether setup is needed and who is signed in
setup(body, request)	UserRequest	New user	Creates the first administrator; refused once a user exists
login(body, request)	LoginRequest	User	Checks the password and locks the name after five failures
logout(request)	HttpServletRequest	Map	Ends the session
users(request), user(id, request)	Optional path: id	List or user	Administrators see everyone; staff see only themselves
create(body, request)	UserRequest	New user	Adds a user; administrators only
update(id, body, request)	Path: id; UserRequest	Updated user	Edits an account; a user changing their own password must give the current one
delete(id, request)	Path: id	Map	Deletes a user; administrators only, and never themselves

AuthFilter functions

Function	Inputs	Output	Purpose
doFilterInternal(...)	Request, response, chain	None	Runs before every request; returns 403 or 401, or passes the request on
sameOrigin(request)	HttpServletRequest	boolean	Compares the Origin header with the Host header
deny(response, status, message)	Response, int, String	None	Writes a JSON error response

Three key rules

price()
Adds a rate for each booked hour. 09:00 up to 18:00 is peak. Takes 10% off when the booking is 4 hours or more.

available()
A slot is taken if an active booking overlaps the time, or a checked-in vehicle has overstayed. A booking ending exactly at the start is allowed.

action()
Runs an UPDATE with the required status. If no row changes, it returns 404 for a missing booking, otherwise 409.

JUnit test functions

Function	Purpose
setup()	Before each test: opens in-memory SQLite, creates the controller and sets a start time two days ahead at 17:00
close()	After each test: closes the database connection
bookingPricingConflictSearchAndCancellation()	Tests pricing, upper-casing, all search types, a conflict, cancellation and rebooking
extensionAndLifecycle()	Tests extension pricing, the 5-hour limit, check-in, check-out and statistics

6. JavaScript Functions and Event Handlers

All browser code is in app.js, with no external library. On load, it attaches listeners, sets the default start time and asks the server who is signed in.

Helper functions

Function	Inputs	Output	Purpose
$(selector)	CSS selector	First matching element	Short form of document.querySelector
money(value)	Number or text	Text such as ₹110.00	Formats rupees with Indian digit grouping and two decimals
escapeHtml(value)	Any value	Safe text	Replaces & < > " ' with HTML entities so data cannot become page HTML
vehicleLabel(value)	Vehicle code	Two-wheeler or Car	Converts the stored code to a label
statusLabel(value)	Status code	Readable status	Converts a status code to a label
roleLabel(value)	Role code	Administrator or Staff	Converts the stored role to a label
isAdmin()	None	boolean	True when the signed-in user is an administrator
notify(message)	Text	None	Shows the toast message and hides it after 5 seconds
pad(n)	Number	Two-digit text	Adds a leading zero when building the default start time

Data and page functions

Function	Inputs	Output	Purpose
api(path, options)	Path after /api, optional fetch options	Promise of parsed JSON	Sends every request, returns to sign-in on a 401, and turns failures into readable errors
parameters()	None; reads the form	Object	Collects startTime, hours and vehicleType
resetAvailability()	None	None	Clears the selected slot and checked values, and hides the result panel
loadStats()	None	Promise	Requests /statistics and fills the four dashboard cards
loadBookings()	None; reads search fields	Promise	Requests bookings, stores them and builds the table rows and buttons
refreshAll()	None	Promise	Runs loadStats() and loadBookings() together, and shows an error if either fails
showReceipt(b)	Booking object	None	Builds the receipt lines and opens the dialog
showAuth(setup)	true for first-run setup	None	Hides the app and shows the sign-in or setup screen
enterApp(user)	User object	None	Shows the app with the user’s name and role, then loads bookings and users
loadUsers()	None	Promise	Requests /users and builds the users table
openUserForm(user)	User, or null for new	None	Fills the form and shows only the fields this user may change
closeUserForm()	None	None	Clears and hides the user form

Action buttons shown by status

Status	Buttons
BOOKED	Receipt, Check in, Cancel; plus Extend if the booking is under 5 hours
CHECKED_IN	Receipt, Check out; plus Extend if the booking is under 5 hours
CHECKED_OUT, CANCELLED	Receipt only

Event handlers

Element	Event	What the handler does
#authForm	submit	Sends sign-in or setup, then opens the app or shows the error on the screen
#logout	click	Ends the session and returns to the sign-in screen
#startTime, #duration, #vehicleType	change	Calls resetAvailability(), so old results cannot be used with new values
#availabilityForm	submit	Requests availability, then shows the price, end time and slot buttons; says so if all slots are taken
#slots	click	Uses event delegation to save and highlight the slot, and enables Confirm
#confirmBooking	click	Requires a registration and slot, posts the booking, then resets and refreshes
#searchForm, #clearSearch	submit, click	Searches, or empties the box and reloads the table
#refresh	click	Calls refreshAll()
#bookingRows	click	Handles Receipt, Check in, Check out, Cancel and Extend for every row
#closeReceipt, #printReceipt	click	Closes the receipt, or opens the print window
#newUser, #cancelUser	click	Opens an empty user form, or closes it
#userForm	submit	Posts a new user, or puts changes to an existing one
#userRows	click	Handles Edit and Delete for every user row
.nav-link	click	Moves the active highlight to the clicked sidebar link

Booking table handler

1. Find the nearest element with data-action. Ignore the click if there is none.
2. Read the booking ID and action, and find the booking in state.bookings.
3. For receipt, call showReceipt() and stop.
4. For extend, show a prompt() for extra hours and validate the answer in the browser.
5. Calculate the estimated new charge with the same rate rules as the server, then ask for confirmation.
6. For other actions, ask for confirmation with confirm().
7. Send POST /bookings/{id}/{action}, show the result as a toast and call refreshAll().

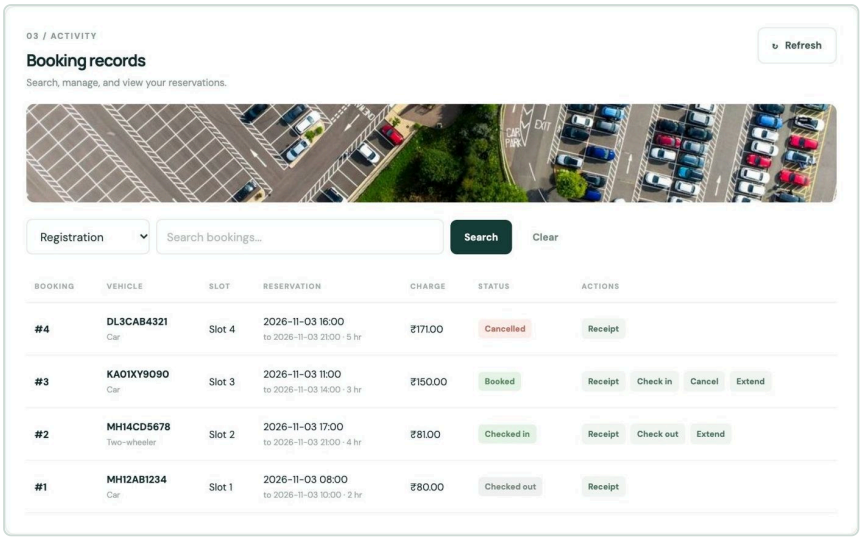


Figure 4: Booking records table with its action buttons

7. Variables and Their Roles

Java fields

Name	Type	Location	Role
db	JdbcTemplate	All three classes	Runs all SQL statements
FORMAT	DateTimeFormatter	ParkingController	Strict uuuu-MM-dd HH:mm format for parsing and storing times
MAX_DAYS_AHEAD	int	ParkingController	Furthest a booking can start: 90 days
ITERATIONS	int	AuthController	PBKDF2 rounds: 210,000
MAX_FAILURES, LOCK_SECONDS	int	AuthController	Five failed sign-ins lock the name for 60 seconds
PUBLIC	String	AuthController	Columns safe to return; leaves out password_hash
random	SecureRandom	AuthController	Makes password salts
failures	Map	AuthController	Failed attempts and lock time for each username
decoy	String	AuthController	Hash checked for unknown names, so they take the same time
USER, OPEN	String, Set	AuthFilter	Request attribute name, and the three routes open without sign-in

Java local variables

Method	Name	Type	Role
addColumn	exists, row	boolean, Map	Whether the column exists; one column description
parseTime, futureHour	e, time	Exception, LocalDateTime	Caught error; the parsed time
registration, compact	text	String	Cleaned registration text
price	total, i, hour, peak	int, boolean	Running total, loop counter, clock hour, peak flag
available, vehicleFree	count	Integer	Number of conflicting rows
find	rows	List	Rows returned for the ID
availability	start, slots, i	LocalDateTime, List, int	Start time; one entry per slot; slot number
book	registration, vehicle, slot, start, end, id	Mixed	Validated fields and the inserted row’s ID
bookings	search, by, pattern, value	String, int	Search text and type, LIKE pattern, number searched
action	changed, now, booking, occupied	int, String, Map, Integer	Rows updated; time; row checked in; other vehicles in the slot
extend	b, status, start, oldEnd, newEnd, totalHours	Mixed	Booking, status, original start, end before and after, new total
login	name, state, rows, password, found, right, next, id	Mixed	Sign-in name, lock state, the row, and the result of the check
update	me, self, target, username, role, name, newPassword, stored	Mixed	Who is acting, the account, its new values and the stored hash

Method	Name	Type	Role
hash, matches	salt, parts, expected	byte[], String[], byte[]	Salt, stored hash parts and expected hash
doFilterInternal	path, user	String, Map	Normalized request path, and the signed-in user

JavaScript shared variables

Name	Kind	Role
state.selectedSlot	Object property	Slot number chosen by the user, or null
state.checkedParameters	Object property	The exact values last checked for availability, reused when booking
state.bookings, state.users	Object properties	Bookings and users currently shown in the tables
state.user	Object property	The signed-in user
state.setup	Object property	True while the first-run setup screen is showing
state.editingUser	Object property	User open in the form, or null for a new user
toastTimer	let variable	Timer ID, used to restart the 5-second toast timer
nextHour	const Date	Next whole hour, used as the default start time
$, money, escapeHtml, vehicleLabel, statusLabel, roleLabel, isAdmin, pad	const functions	Shared helper functions

JavaScript local variables

Function or handler	Name	Role
notify	toast	The toast element
api	response, result	The fetch response, and its parsed JSON
loadStats	stats	Dashboard values from the server
loadBookings	search, params, rows	Search text, query text and table body
loadBookings	b, id, status, actions	Current booking, ID, status, and HTML for its buttons
Availability handler	params, result, s	Values sent, the response, one slot entry
Slot and Confirm handlers	button, registration, booking	Clicked button, registration and returned booking
showReceipt	lines	Label and value pairs for the receipt
Table handler	button, id, action, booking, body	Clicked button, its booking and the request body
Table handler	answer, extraHours, start, newPrice, i, hour, peak, result	Extension prompt, estimate loop and returned booking
Sign-in handler	button, body, user	Submit button, values sent and returned user
loadUsers	id, self, remove	User ID, whether it is the signed-in user, Delete button HTML
openUserForm	self	True when editing your own account
User form handler	editing, body, saved	User being edited, values sent, returned user
Users table handler	button, id, user	Clicked button and its user

8. API Endpoints and Communication

All endpoints use JSON. Errors include readable text in a message field. Every route needs a signed-in user except the three marked Open, and other websites receive 403 on anything but GET.

Booking routes

Method and path	Input	Success	Errors
GET /api/availability	Query: startTime, hours, vehicleType	200: slots, price, endTime	400 invalid input
POST /api/bookings	Body: registration, vehicleType, startTime, hours, slot	200: new booking	400 invalid; 409 slot taken or vehicle already booked
GET /api/bookings	Query: search, by	200: list of bookings	400 invalid search
GET /api/bookings/{id}	Path: id	200: one booking	404 not found
POST /api/bookings/{id}/cancel	Path: id	200: updated booking	404; 409 not allowed
POST /api/bookings/{id}/check-in	Path: id	200: updated booking	404; 409 not allowed or slot occupied
POST /api/bookings/{id}/check-out	Path: id	200: updated booking	404; 409 not allowed
POST /api/bookings/{id}/extend	Path: id; body: extraHours	200: updated booking	400 invalid; 404; 409 unavailable
GET /api/statistics	None	200: totals	401 if signed out

Sign-in and user routes

Method and path	Input	Success	Errors
GET /api/auth/status **Open**	None	200: setupNeeded, user	None
POST /api/auth/setup **Open**	Body: username, fullName, password	200: new administrator	400 invalid; 409 already set up
POST /api/auth/login **Open**	Body: username, password	200: user	401 incorrect; 429 locked
POST /api/auth/logout	None	200: signedOut	401 if signed out
GET /api/users	None	200: list of users	401
GET /api/users/{id}	Path: id	200: one user	403; 404
POST /api/users	Body: username, fullName, password, role	200: new user	400; 403; 409 name taken
PUT /api/users/{id}	Path: id; body: username, fullName, role, password, currentPassword	200: updated user	400; 403; 404; 409
DELETE /api/users/{id}	Path: id	200: deleted	403; 404; 409 own account

The by search value is registration, id or slot. Registration search matches part of the number and ignores case. The page never calls GET /api/bookings/{id} or GET /api/users/{id}.

How the browser talks to the server

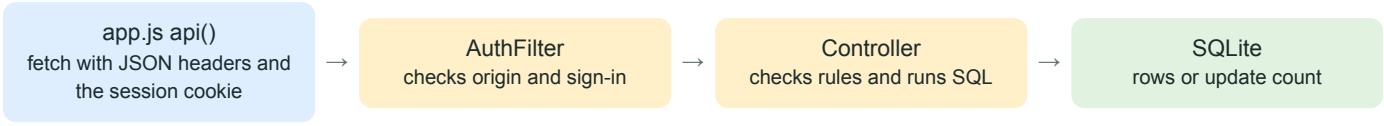


Figure 5: One request from the browser to the database. The JSON response, or an error with a message, returns the same way.

api() is the single function that calls fetch. It adds the JSON header, turns failures into readable errors, and returns the parsed JSON. Handlers then update the page, or show the error with notify().

```
POST /api/bookings/1/extend
{ "extraHours": 1 }

Response: { "id": 1, "car_number": "MH12AB1234", "slot": 2, "hours": 3, "price": 110.0,
"status": "BOOKED", ... }
```

9. Application Flows

Booking

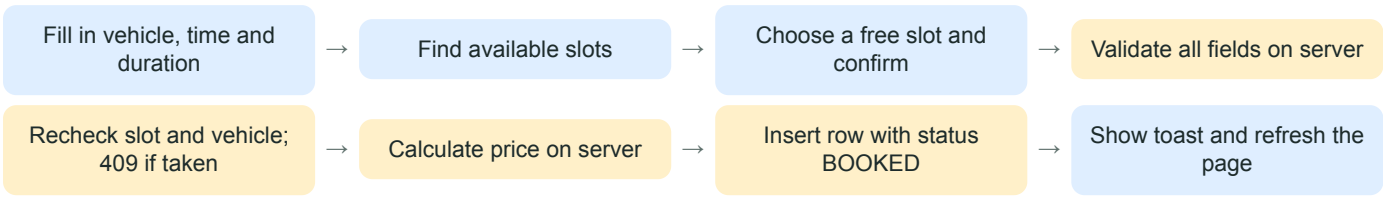


Figure 6: Booking flow. Blue steps run in the browser, yellow on the server

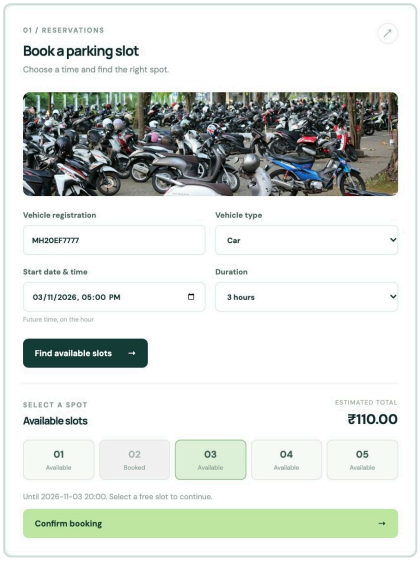
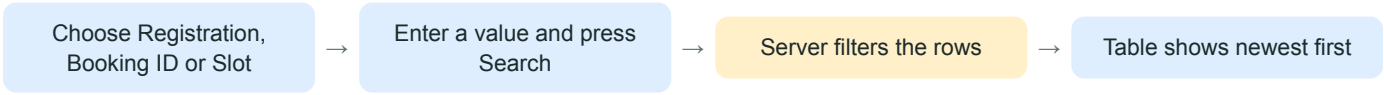


Figure 7: Availability result with a slot selected

The server repeats every check when the booking is confirmed, because the slot could be taken after the availability check.

Search



An empty search lists every booking. ID and slot searches must be whole numbers, otherwise the server returns 400.

Pricing

Each hour is charged at the rate in force when it starts. Bookings of 4 or 5 hours get 10% off the whole charge. Peak is 09:00 up to 18:00. All other hours are off-peak.

Car

Peak ₹50 per hour
Off-peak ₹30 per hour

Two-wheeler

Peak ₹30 per hour
Off-peak ₹20 per hour

Worked examples, confirmed by testing

Booking	Hourly rates	Subtotal	Discount	Charge
Car, 08:00, 2 hours	30 + 50	₹80	None	₹80.00
Car, 17:00, 3 hours	50 + 30 + 30	₹110	None	₹110.00

Booking	Hourly rates	Subtotal	Discount	Charge
Two-wheeler, 17:00, 4 hours	30 + 20 + 20 + 20	₹90	10%	₹81.00
Car, 16:00, 5 hours	50 + 50 + 30 + 30 + 30	₹190	10%	₹171.00

Check-in

Press Check in and confirm → Server checks no other vehicle is in the slot → Update requires BOOKED → Status becomes CHECKED_IN

Check-in is not limited to the reserved period, so a booking can be checked in straight after it is made. It returns 409 if another vehicle is still in the slot.

Check-out

Press Check out and confirm → Update requires CHECKED_IN → Time saved in checked_out_at → Slot becomes free at once

The charge does not change for an early or late departure. It moves from outstanding to completed in the statistics, which is when Completed charges goes up.

Cancellation

Press Cancel and confirm → Update requires BOOKED and a future start → Status becomes CANCELLED, and the time is free again

Rules at a glance

Action	Allowed when	Otherwise
Book	Slot free, vehicle has no overlapping booking, valid fields	400 or 409
Check in	BOOKED, slot not occupied	409
Check out	CHECKED_IN	409
Cancel	BOOKED, start time still ahead	409
Extend	BOOKED or CHECKED_IN, end time still ahead, total up to 5 hours, added time free for the slot and vehicle	400 or 409

Extension

Prompt for extra hours → Show estimate and confirm → Check status, end time and 5-hour limit → Check only the added period → Save end time, hours and new full price

Figure 8: Extension flow

The server checks only the added period and ignores the booking's own row. The whole price is recalculated from the original start, so reaching 4 hours applies the discount to every hour.

Receipt and statistics

The receipt uses the booking already loaded in the page and states that no payment is processed. The dashboard shows total, checked_in, completed and completed_charges. The API also returns cancelled and outstanding_charges, which the page does not display.

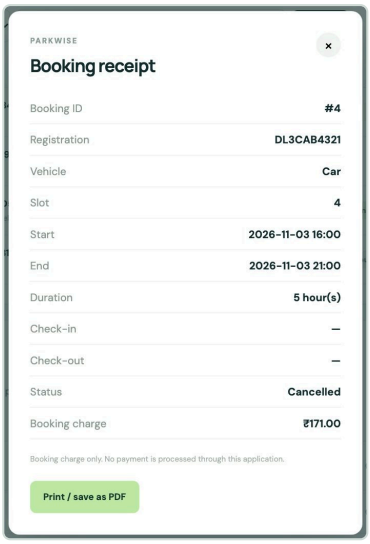


Figure 9: Booking receipt

10. Login and User Management

Parkwise has no default password. The first person to open a new installation creates the administrator account. After that, everyone signs in, and the administrator manages the other accounts.

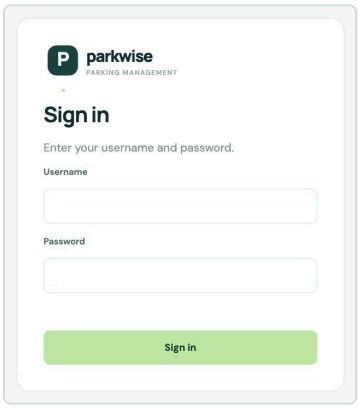


Figure 11: Sign-in screen

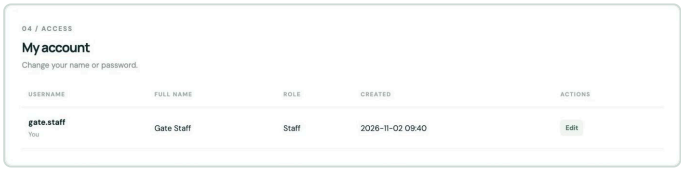


Figure 12: My account page for a staff user

Roles

Task	Administrator	Staff
Bookings, check-in, check-out, search, receipts	Yes	Yes
Create a user	Yes	No
View users	Everyone	Own account only
Edit a user	Anyone: username, name, role, password	Own name and password only
Delete a user	Anyone except themselves	No

User CRUD

Operation	Route	How it works on the screen
Create	POST /api/users	New user opens an empty form. The username must be unused and the password at least 8 characters.

Operation	Route	How it works on the screen
Read	GET /api/users	The Users page lists accounts. The password hash is never sent to the browser.
Update	PUT /api/users/{id}	Edit fills the form. Leaving the password blank keeps the current one.
Delete	DELETE /api/users/{id}	Delete asks for confirmation. Bookings are not affected.

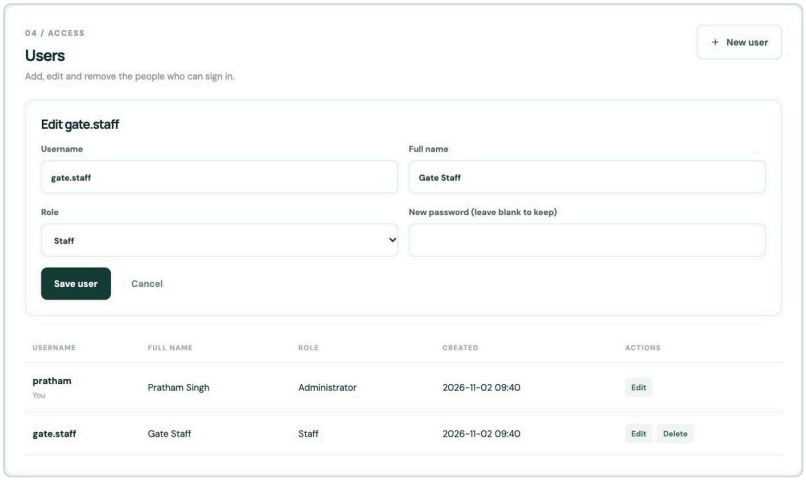


Figure 13: Users page with the edit form open

Sign-in

Enter username and password → Refuse if the name is locked → Compare hash in constant time → New session, cookie sent → Open the app

Figure 14: Sign-in flow

How accounts are protected

Hashed passwords
PBKDF2-HMAC-SHA256, 210,000 rounds, with a 16-byte random salt

Lockout
Five failed attempts lock that name for one minute

Same message
A wrong name and a wrong password look the same

Session rules
HttpOnly, SameSite=Lax cookie with a new ID at each sign-in

Immediate removal
Deleting a user, or resetting their password, ends their session

Safe changes
Your own password needs the current one. The last administrator cannot be removed or demoted

Errors found and fixed during testing

Problem found	Fix
A huge extension value overflowed the total and set the charge to ₹0	The extra hours are limited to 1 to 4 before anything is added
The same vehicle could hold two slots at once	vehicleFree() refuses overlapping bookings, ignoring spaces and hyphens
Registrations accepted symbols and code-like text	registration() allows only letters, numbers, spaces and hyphens
Bookings were allowed decades ahead, even in year 9999	Bookings can start at most 90 days ahead
Search needed the full registration	Search now matches part of a registration
Decimal hours such as 1.5 were accepted	The server refuses a decimal where a whole number is needed

Problem found	Fix
Another website could cancel a booking	AuthFilter refuses other origins, and the cookie is SameSite
Anyone on the computer could read all bookings	Every route now needs sign-in, apart from the three that start it
The page gave no clear message when all slots were full	The availability note now says so
Sign out was hidden on small screens	It moved to the top bar, which is always visible

11. Running the Project

Java 17 or later
Maven is not needed to run the jar

A web browser
Open the local address

Port 8080 free
Or choose another port

Run the application

Open a terminal in the ParkwiseWeb folder and run:

```
java –jar parkwise.jar
```

Open http://localhost:8080. On first use, create the administrator account. Keep the terminal open, and press Ctrl+C to stop. If port 8080 is in use, run:

```
java –jar parkwise.jar ––server.port=8081
```

The first run creates parking.db in the folder the command is run from. Start from the same folder to keep your records. Sign-in sessions end after 30 minutes without activity or when the server restarts.

Build from source

These commands from the README need JDK 17 or later, Maven and internet access. Maven was not installed on the test computer, so they were not run.

```
mvn test
mvn package
java –jar target/parking–web–1.0.0.jar
```

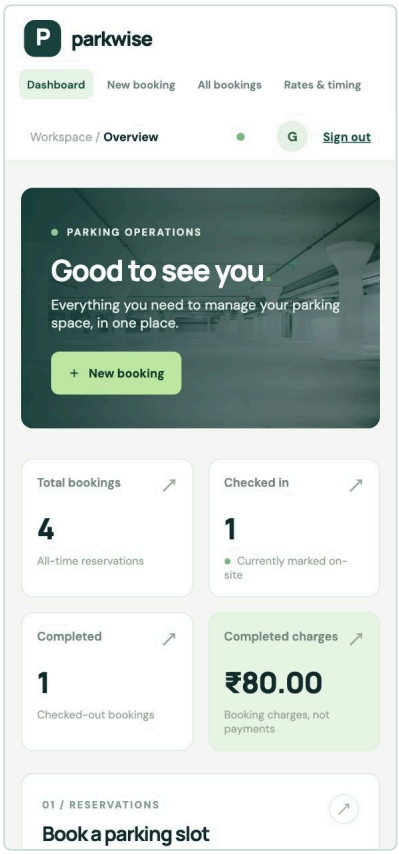


Figure 15: Parkwise on a phone, at 390 pixels wide. The records table has its own scrolling area.

12. Testing

77	94	34	0
Login and access checks	Booking API checks	Screen checks	Final failures

All 205 checks ran against the final jar, with Java 21 and Google Chrome on macOS, using separate database files. They were written for this report and are not part of the project.

Area	Confirmed behaviour
Access	Signed-out users are refused on every protected route; 10 path tricks were tried to step round the check and all failed
Sign-in	Wrong, blank and SQL-injection logins are refused; five failures lock the name; a made-up session is refused
User CRUD	Create, read, update and delete work; duplicates, bad roles and short passwords are refused
Roles	Staff cannot view, create, edit or delete others, or promote themselves
Safeguards	Deleted users and reset passwords end sessions at once; the last administrator cannot be removed
Other websites	Requests from another origin receive 403, signed in or out
Validation	Rejected bad times, dates, durations, vehicles, registrations, slots, decimals and malformed JSON
Conflicts	Refused overlaps and a vehicle in two slots; accepted back-to-back bookings; 20 simultaneous requests gave one booking
Pricing	Confirmed peak, off-peak, boundary, discount and midnight-crossing charges
Search	Confirmed registration, part-registration, ID and slot search; SQL injection found nothing
Extension	Confirmed the recalculated price, the 5-hour limit, a blocked extension and a huge value being refused
Check-in and check-out	Confirmed check-in straight after booking, check-out, repeated actions being refused, and the overstay block
Screens	Confirmed sign-in, booking, receipt, extend, check-in, check-out, cancel, users and the charge total updating
Safety	HTML typed into a user’s name showed as plain text
Upgrade	A copy of an existing database opened in the new version with its bookings intact
Phone width	No sideways page scrolling at 390 pixels wide, with Sign out visible

How the tests were run

- Java source was compiled with —release 17.
- Changed classes were placed in the jar with the jar tool, not rebuilt with Maven.
- Some start and end times were moved in test databases to test overstays and ended bookings.
- The 2 JUnit tests in the project passed on the earlier version. They were not rerun on the final build, because the JUnit tools were not installed.

Not tested

- Many users at once, or two server processes
- Large numbers of bookings or users
- Time zone or daylight-saving changes
- Other browsers and full phone use
- Printing a receipt
- Maven build commands
- The lockout expiring after one minute
- Sessions timing out after 30 minutes

13. Limitations

No payment
No payments, refunds or late fees. It uses plain HTTP, so do not expose it publicly.

Password recovery
There is no email reset. If the only administrator forgets their password, there is no way back in from the screen.

Fixed rules
Five slots and all rates are written in code. Whole-hour bookings of 1 to 5 hours only.

Shared slots
Cars and two-wheelers use the same five slots.

No editing
Bookings cannot be edited or deleted after creation.

Charges do not adjust
No refund for early check-out and no extra charge for an overstay.

No-shows
Unused bookings stay BOOKED, with no automatic expiry.

Local time
Uses the computer clock, with no time zone or daylight-saving handling.

One process
Only one server instance should use a database.

Simple code structure
Controllers hold their own logic. Pricing is repeated in Java and JavaScript.

Data rules in Java
The database has no CHECK constraints, extra indexes or link between users and bookings.

Online fonts
Google Fonts need internet. The photos are stored locally.

14. Conclusion

Parkwise meets its objectives as a local parking management prototype. It controls who can sign in, manages user accounts, shows availability, prevents overlapping bookings, calculates charges, tracks each booking’s status, and keeps its records in a database. Its strongest design choice is that the server rechecks every rule, even when the browser has already checked it.

All 205 checks on the final build passed, which confirms the tested behaviour on one computer. It would need HTTPS, payment handling and stronger data rules before real use.

Future work: add password reset, record which user made each booking, move rates and slots into settings, keep pricing in one place, split the controllers into layers, add database constraints, handle time zones, and automate no-show handling.

Photos in the application are from Unsplash: Claudio Schwarz, Benjamin Cheng, Sara Kurfess and Deski Jayantoro.